📋 Таска: REST API для `Message` — GET (список с пагинацией по 40) и POST (создание)

🎯 Цель

Отдать два эндпоинта:

- `GET /api/messages/` — список сообщений, **новые сверху**, строго по **40 на страницу**, с метаданными пагинации;
- `POST /api/messages/` — создать сообщение.

Использовать сервисные функции `create_message()` и `get_messages()` из Т-БД, чтобы HTTP-слой не лез в ORM напрямую. Это заодно проверит, что слой данных спроектирован правильно.

Зависит от: Т-БД (модель `Message` + `services.py`) и Т-CORS (уже настроен).

🧠 Как DRF отдаёт пагинацию

Когда в DRF включена `PageNumberPagination`, ответ на `GET` — это **не плоский массив**, а объект:

```
{
  "count": 51,
  "next": "http://127.0.0.1:8000/api/messages/?page=2",
  "previous": null,
  "results": [
    { "id": 51, "object": "...", "text": "...", "date": "2026-11-09T12:00:00Z" },
    ...
  ]
}
```

- `count` — сколько всего записей;
- `next` / `previous` — **полные URL** соседних страниц (или `null`, если их нет);
- `results` — массив сообщений на текущей странице.

Фронт будет использовать `next` как готовую ссылку (`fetch(data.next)`) — **не склеивать вручную с `API_BASE`**.

🛠 Шаг 1: Настроить `REST_FRAMEWORK` с пагинацией

Открой `backend/config/settings.py`. Найди `REST_FRAMEWORK` (он уже есть после таски для `Post`) и **дополни** блок:

```
REST_FRAMEWORK = {
```

```
    "DEFAULT_AUTHENTICATION_CLASSES": [],
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
    "DEFAULT_PAGINATION_CLASS": "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 40,
}
```

- **`DEFAULT_PAGINATION_CLASS`** — включает пагинацию по умолчанию для **всех** ViewSet'ов, у которых не переопределён `pagination_class`.
- **`PAGE_SIZE: 40`** — размер страницы по умолчанию. Ровно 40, как договаривались.

> ⚠️ **Внимание, это повлияет и на `Post`!** `GET /api/posts/` начнёт возвращать `{count, next, previous, results}` вместо плоского массива. Это **ок**, но фронт-команда должна быть предупреждена — у них в T2.2 `renderPosts(posts)` ожидает массив. Нужно либо:
>
> - **Согласовать с фронтом**: они обновят T2.2 под новый формат (правильнее, потому что пагинация им всё равно нужна в T2.6);
> - Либо на `PostViewSet` явно отключить пагинацию (`pagination_class = None`), а на `MessageViewSet` — включить.
>
> **Рекомендую первое**: включить глобально и попросить фронт сразу адаптироваться. Меньше расхождений потом.

🛠 Шаг 2: Сериализатор

Создай `backend/messaging/serializers.py`:

```
from rest_framework import serializers

from .models import Message


class MessageSerializer(serializers.ModelSerializer):
    class Meta:
        model = Message
        fields = ["id", "object", "text", "date"]
        read_only_fields = ["id", "date"]

    def validate_object(self, value):
        value = value.strip()
        if not value:
            raise serializers.ValidationError("Поле 'object' не может быть пустым.")
        return value

    def validate_text(self, value):
        value = value.strip()
        if not value:
            raise serializers.ValidationError("Поле 'text' не может быть пустым.")
        return value
```

- **`read_only_fields = ["id", "date"]`** — клиент не может подделать `id` или дату; `date` ставится сервером (`auto_now_add`).
- **`validate_*`** — тримминг и отклонение пустых строк. Это **дублирует** проверки в `create_message()` , но это нормально: сериализатор отвечает за валидацию входных данных, сервис — за инварианты. Если кто-то вызовет `create_message()` из shell, сервис тоже отклонит пустоту.

🛠 Шаг 3: ViewSet

Открой `backend/messaging/views.py` (или создай, если пусто):

```
from rest_framework import mixins, status, viewsets
from rest_framework.response import Response

from .serializers import MessageSerializer
from .services import create_message, get_messages


class MessageViewSet(
    mixins.ListModelMixin,
    mixins.CreateModelMixin,
    viewsets.GenericViewSet,
):
    serializer_class = MessageSerializer

    def get_queryset(self):
        # queryset нужен DRF для schema/browsable API.
```

```
        # Реальную пагинацию делает get_messages() в list().
        from .models import Message
        return Message.objects.all()

    def list(self, request, *args, **kwargs):
        page_number = request.query_params.get("page", 1)
        page = get_messages(page=page_number)

        serializer = self.get_serializer(page.object_list, many=True)

        paginator = page.paginator
        base_url = request.build_absolute_uri(request.path)

        next_url = None
        prev_url = None
        if page.has_next():
            next_url = f"{base_url}?page={page.next_page_number()}"
        if page.has_previous():
            prev_url = f"{base_url}?page={page.previous_page_number()}"

        return Response({
            "count": paginator.count,
            "next": next_url,
            "previous": prev_url,
            "results": serializer.data,
        })

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        message = create_message(
            object=serializer.validated_data["object"],
            text=serializer.validated_data["text"],
        )

        out = self.get_serializer(message)
        return Response(out.data, status=status.HTTP_201_CREATED)
```

Разберём по частям.

Почему `mixins.ListModelMixin, mixins.CreateModelMixin` вместо `ModelViewSet`:

- `ModelViewSet` дал бы ещё `retrieve`, `update`, `partial_update`, `destroy` — а анонимная борда этого не должна позволять.
- Только `list` + `create` — и никаких случайных методов.

Почему переопределён `list()` вместо `pagination_class` на классе:

- Мы хотим прогнать queryset **через `get_messages()`** — то есть через `Paginator`, который уже написан и протестирован.
- Иначе получилось бы **две параллельные системы пагинации**: одна в `services.py`, другая в DRF. Это дублирование, и при изменении `PAGE_SIZE` пришлось бы менять в двух местах.

- Переопределив `list()`, мы имеем **один источник правды** — `get_messages()` — и вручную формируем ответ в формате, который ожидает фронт (`count/next/previous/results`).

Что возвращает `list()`:

- `page.object_list` — сообщения текущей страницы, сериализуем через `MessageSerializer(many=True)`;
- `page.paginator.count` — всего сообщений;
- `request.build_absolute_uri(request.path)` — полный URL вида `http://127.0.0.1:8000/api/messages/` (используется как база для `next` / `previous`);
- `page.has_next()`, `page.next_page_number()` — из `Paginator`. Собираем URL вручную в формате, **идентичном** тому, что отдаёт DRF при `PageNumberPagination` — фронт не заметит разницы.

Почему `create()` переопределён, а не `perform_create`:

- Нам нужно передать данные в **сервисную функцию** `create_message()`, а не создавать через `serializer.save()`.
- `serializer.save()` вызвал бы `Message.objects.create()` напрямую в обход `services.py` — теряем централизацию и `@transaction.atomic`.
- Отдельный `create()` — прозрачнее: валидировали → вызвали сервис → сериализовали результат → вернули `201`.

Почему `get_queryset()` всё равно нужен:

- DRF использует его для browsable API и для schema-генерации. Без него форма в браузере и `/api/messages/` в browsable mode могут падать.
- Реальную пагинацию делает `list()` — `get_queryset()` только «объявляет» queryset.

🛠 Шаг 4: URL'ы

Открой `backend/messaging/urls.py` (создай, если нет):

```
from rest_framework.routers import DefaultRouter

from .views import MessageViewSet

router = DefaultRouter()
router.register(r"messages", MessageViewSet, basename="message")

urlpatterns = router.urls
```

Подключи в `backend/config/urls.py`:

```
from django.contrib import admin
```

```
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("crackerchat.urls")),
    path("api/", include("messaging.urls")),  # ← добавили
]
```

> 💡 **Два `include` под одним префиксом `api/`** — это нормально. Django складывает URL-паттерны из обоих роутеров: `/api/posts/` из `crackerchat`, `/api/messages/` из `messaging`. Порядок не важен, пока паттерны не конфликтуют.

Перезапусти сервер после правок `settings.py` и `urls.py`:

```
python manage.py runserver
```

✅ Проверка

1. GET — пустой список (если в БД ещё нет сообщений):

```
curl "http://127.0.0.1:8000/api/messages/"
```

Ожидается:

```
{"count": 0, "next": null, "previous": null, "results": []}
```

2. POST — создание:

```
curl -X POST http://127.0.0.1:8000/api/messages/ \
  -H "Content-Type: application/json" \
  -d '{"object":"Привет","text":"первое сообщение"}'
```

Ожидается `201 Created`:

```
{"id": 1, "object": "Привет", "text": "первое сообщение", "date": "2026-11-09T12:34:56.789Z
```

3. GET — теперь не пусто:

```
curl "http://127.0.0.1:8000/api/messages/"
```

Ожидается `count: 1`, `results` с одним сообщением.

4. Пагинация — строго 40 на страницу:

Быстро набей 50+ сообщений через shell:

```
python manage.py shell -c "
from messaging.services import create_message
for i in range(50):
    create_message(object=f'Тема {i}', text=f'Текст {i}')
"
```

Затем:

```
curl "http://127.0.0.1:8000/api/messages/" | python -m json.tool | grep -E '"count"|"next"|
```

Ожидается:

- `"count": 51` (1 + 50);
- `"next": "http://127.0.0.1:8000/api/messages/?page=2"` ;
- `"previous": null` ;
- `results` длиной **ровно 40**.

Перейди на вторую страницу:

```
curl "http://127.0.0.1:8000/api/messages/?page=2"
```

- `results` длиной 11 (51 – 40);
- `next = null` ;
- `previous = http://127.0.0.1:8000/api/messages/?page=1` .

5. Порядок — новые сверху: В `results` на странице 1 первым должен быть `id: 51` (последнее созданное), последним — `id: 12` .

6. Невалидная страница — возвращает первую:

```
curl "http://127.0.0.1:8000/api/messages/?page=abc"
```

Ожидается `200` и первая страница (`get_page` не бросает исключение на мусор).

7. Пустой текст отклоняется:

```
curl -X POST http://127.0.0.1:8000/api/messages/ \
  -H "Content-Type: application/json" \
  -d '{"object":"x","text":"   "}'
```

Ожидается `400` с `{"text": ["Поле 'text' не может быть пустым."]}` .

8. PUT / DELETE запрещены:

```
curl -X DELETE "http://127.0.0.1:8000/api/messages/1/"
curl -X PUT "http://127.0.0.1:8000/api/messages/1/" -d '{}' -H "Content-Type: application/j
```

Оба — `405 Method Not Allowed`.

9. Browsable API: Открой `http://127.0.0.1:8000/api/messages/` в браузере — DRF покажет форму и список.

10. CORS на месте:

```
curl -I -H "Origin: http://127.0.0.1:3000" "http://127.0.0.1:8000/api/messages/"
```

В ответе должен быть `Access-Control-Allow-Origin: http://127.0.0.1:3000`.

11. Фронт может забрать: В консоли браузера на `http://127.0.0.1:3000`:

```
fetch('http://127.0.0.1:8000/api/messages/').then(r => r.json()).then(console.log)
```

Должен вывести объект `{count, next, previous, results}`.

🚫 Подводные камни

- **`KeyError: 'object' в create()`** — сериализатор не пропустил поле. Проверь, что `object` и `text` есть в `fields`, и что клиент их прислал.
- **`GET /api/messages/` возвращает плоский массив вместо `{count, next, ...}`** — не переопределён `list()`, или `list()` упал и DRF откатился к `ListModelMixin` (посмотри стектрейс).
- **`next` содержит `http://testserver/...`** — это `request.build_absolute_uri` в тестах. В браузере/curl будет правильный хост. Если важно — передавай явный хост, но для dev это не проблема.
- **`page=0` или `page=-1`** — `get_page()` вернёт первую страницу (защита от невалидного ввода). Это ожидаемое поведение.
- **`page=9999` (за пределами)** — тоже вернёт первую страницу, не `404`. Если нужна строгая семантика — используй `paginator.page()` вместо `get_page()` и обрабатывай `EmptyPage`, но для MVP избыточно.
- **CORS не работает на POST** — проверь, что в `CORS_ALLOWED_ORIGINS` и `CSRF_TRUSTED_ORIGINS` есть `http://127.0.0.1:3000` и `http://localhost:3000`. У нас `DEFAULT_AUTHENTICATION_CLASSES: []`, поэтому CSRF-токен не нужен, но CORS-заголовки всё равно обязательны.
- **Пагинация `Post` сломалась** — если включил `PAGE_SIZE` глобально, `GET /api/posts/` теперь возвращает `{count, ...}` вместо массива. **Предупреди фронт до мержа.**
- **`Message.objects.all()` в `get_queryset` без импорта** — если выносишь `Message` в

`get_queryset`, не забудь `from .models import Message` наверху файла (в примере выше импорт внутри метода — это хак для избежания циклического импорта, но лучше сделать обычный импорт наверху, цикла тут нет).

- **`serializer.data` в `list()` возвращает `ReturnList`** — это нормально, DRF сам сериализует в JSON при рендере. Не пытайся `json.dumps` вручную.

📝 Критерии готовности (Definition of Done)

- [ ] `REST_FRAMEWORK` в `settings.py` содержит `DEFAULT_PAGINATION_CLASS` и `PAGE_SIZE: 40`.
- [ ] `messaging/serializers.py` с `MessageSerializer` и валидацией `object` / `text` (тримминг, непустота).
- [ ] `messaging/views.py` с `MessageViewSet` (только `list + create`).
- [ ] `list()` вызывает `get_messages()` из `services.py` (не `Message.objects.all()` напрямую в ответе).
- [ ] `create()` вызывает `create_message()` из `services.py` (не `serializer.save()`).
- [ ] Ответ `GET /api/messages/` — `{count, next, previous, results}`, `results` строго по **40**.
- [ ] `next` / `previous` — полные URL; на первой странице `previous: null`, на последней `next: null`.
- [ ] Сообщения отсортированы **новые сверху**.
- [ ] `POST` возвращает `201` с созданным объектом (включая `id` и `date`).
- [ ] `POST` с пустым `object` или `text` возвращает `400`.
- [ ] `PUT` / `DELETE` / `PATCH` / `GET /api/messages/<id>/` возвращают `405`.
- [ ] CORS: `curl -I -H "Origin: http://127.0.0.1:3000"` возвращает `Access-Control-Allow-Origin`.
- [ ] Фронт может сделать `fetch` на `/api/messages/` без ошибок.
- [ ] Фронт-команда **уведомлена** об изменении формата `/api/posts/` (если включил пагинацию глобально).
- [ ] Изменения закоммичены.